

Multi-Agent AI for Smart Contract Audit and Cybersecurity Risk Assessment

A source-review platform for identifying potential vulnerabilities, inspecting evidence and documenting security decisions.

THE CONTEXT

Code governs the asset

Smart contracts encode permissions, transfers and interactions in digital asset workflows. A defect in those controls can affect custody, transaction integrity or availability.

THE REVIEW PROBLEM

A finding needs evidence

A code pattern can be benign in context; a plausible AI explanation can still be wrong. Reviewers need the exact source, a clear rationale and a way to trace each claim.

THE PLATFORM RESPONSE

Make findings inspectable

Combine a reproducible static baseline with optional Bedrock reasoning. Preserve the source record and keep model findings identifiable for human review.

CONNECT TECHNICAL FINDINGS TO DIGITAL ASSET EXPOSURE

Review question	Potential exposure	Evidence to inspect
Who can change or move value?	Unauthorized actions or asset transfers	Caller checks, state changes and recipients
What happens across a contract call?	Repeated withdrawals or inconsistent state	Call order, state updates and return values
Which assumptions drive outcomes?	Manipulated payouts or unavailable functions	Inputs, timing, randomness and iteration

One workspace. Two analysis modes.

Inspect a single Solidity file, investigate candidate findings and export a source-linked report.

STATIC ANALYSIS

Run checks on-device

15 pattern rules examine the submitted source directly in the browser.
No API key or backend is required.

The public website and local workspace both support this mode. The source remains in the browser.

API ANALYSIS

Add Bedrock reasoning

Enter your Bedrock key in the website. Its server sends the source and static baseline to GPT-6 Astra.

The user starts the request explicitly. Static and model findings remain distinct, and model evidence is checked before acceptance.

WORKSPACE FUNCTIONS

Task	Interaction	What the reviewer receives
Prepare the source	Upload .sol, edit code or open a reference case	A numbered source snapshot for inspection
Investigate findings	Filter by severity, search and expand a finding	Rationale, source evidence and recommended action
Inspect & share	Jump to source lines; export JSON or Markdown	Source hash, analysis metadata and accepted findings

API review uses your key through the website server. Keys stay in tab/request memory; AWS usage is charged to your account.

Prioritize risk. Preserve the context.

The static baseline flags recognizable patterns. Each candidate still needs contextual review.

Risk family	Signals examined	Questions for the reviewer
Access control & custody	tx.origin authorization, ownership changes and asset-transfer patterns	Who is permitted to act? Are recipients and state changes consistent with intended business rules?
External interactions	Calls before balance updates, unchecked call/send results and caller-controlled callbacks	Can control re-enter the contract? What state is visible? How are failures handled?
Execution authority	delegatecall and selfdestruct expressions	Who controls the target? What authorization, storage and chain-specific assumptions apply?
Economic & time assumptions	Block-data randomness, predictable game outcomes and timestamp-sensitive logic	Can a participant influence inputs, timing or payouts? What information is already public?
Configuration & availability	Legacy or missing compiler declarations and potentially unbounded iteration	Which compiler is intended? Can execution grow beyond a usable gas budget?

Severity is a proposed prioritization aid. A matched expression is not proof of an exploitable vulnerability.

Every conclusion should be traceable.

Read the claim, inspect the cited lines and assess the proposed action against the contract's intended behavior.

ILLUSTRATIVE FINDING / STATIC

External call before balance update

A call before the balance update can leave state exposed during an external interaction. The reviewer must determine whether re-entry is possible and economically meaningful.

```
(bool ok, ) = msg.sender.call{value: amt}("");
require(ok, "transfer failed");
balances[msg.sender] -= amt;
```

Review action

Check the call path and guards. Consider updating state before the call or using an appropriate reentrancy guard; validate the change with targeted tests.

REPORT CONTENTS

Finding	Proposed severity, origin, category, rationale and recommended action.
Evidence	Source line range and excerpt linked to the submitted source.
Provenance	Filename, source SHA-256, run metadata and model identity when used.

VALIDATION & REVIEW BOUNDARIES

Model output must satisfy the report schema and quote source excerpts that match the submitted file. Invalid model output is rejected; the static baseline remains available.

This workflow does not compile contracts, resolve imports, execute exploits, modify code or deploy on-chain. Human review is needed to determine exploitability and remediation.

Based on the supplied Duke FinTech / DTCC presentation and the platform's implemented behavior. No performance or loss-reduction claims are implied.